AntiVAX Hardware Specification

Tristan Miller, Rose Novack, Robert White

March 2026

Hardware Component Descriptions

All registers, unless otherwise stated are 32 bit long.

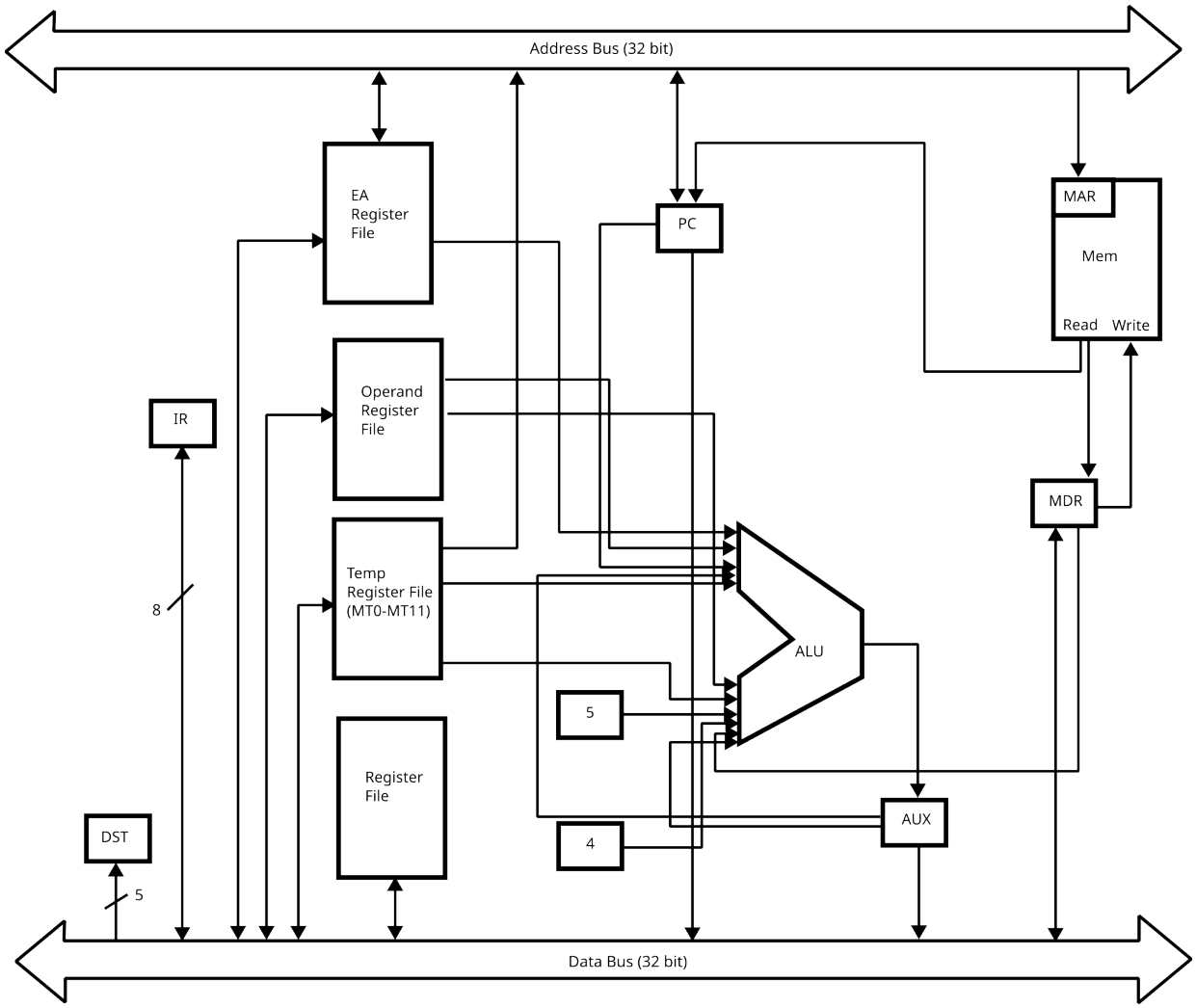
Micro Control Unit Components:

- μMem - Micro Random Access Memory - 16 bit addresses. Each unit is 45 bits.
- μPC - Micro Program Counter
- μIR - Micro Instruction Register
- μRA - Micro Return Address Register - Used in address mode decoding.
- μA - Micro Arg Register - Used by some micro code subroutines to pass arguments. Only 5 bits.
- ALU for μMem address calculation.

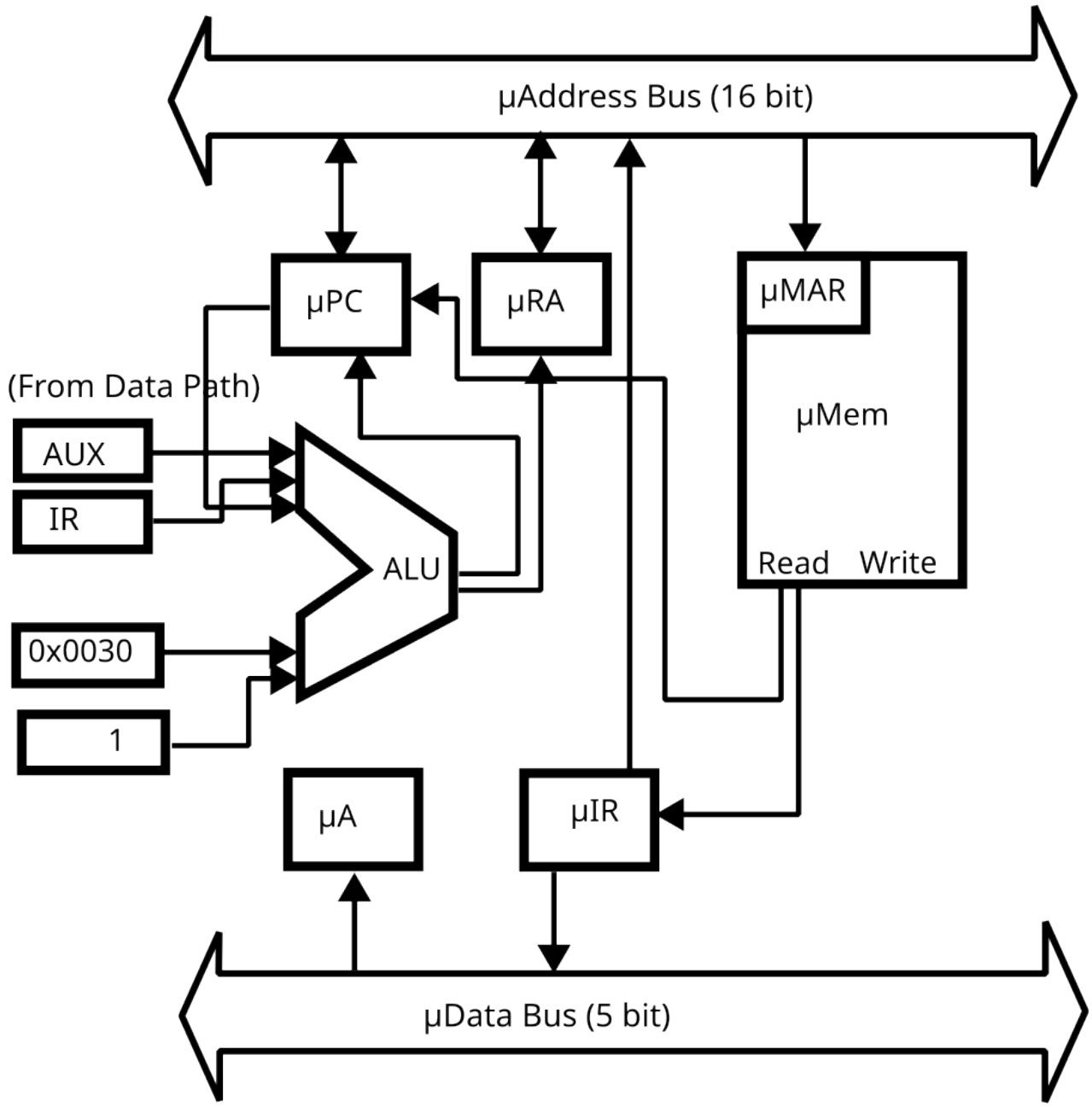
Data Path Components:

- regfile - Register File (31 Registers) (program counter not included)
- PC - Program Counter
- IR - Instruction Register. Only 8 bits.
- Arithmetic/Logic Unit
- EA - Effective Address Register File (6 Registers)
- orf - Operand Register File for holding the data of all address modes (6 Registers)
- 12 Temporary Registers (MT0 - MT11)
- MDR - Memory Data Register (used for reading/writing data)
- AUX - Auxiliary register to hold ALU results since we can't write to a bus directly
- Mem - Main memory. 32 bit addresses that are byte addressable. Can read four units at the same time. Loops (there is no memory overflow).
- DST - 3 bit Destination Register - 0 if dest is a memory address (location is in ea), otherwise number in regfile if it's a register

Data Path Diagram:



Micro Controller Diagram



Register Transfer Language

Since address modes will result in different possible destinations and origins. The following notation will be used in this table:

- data(opX) - The value pointed to by opX (i.e. an immediate or the value at that memory location). This can be read from the operand register file.
- dest(opX) - The destination pointed to by opX (cannot be immediate address mode). Each instruction at most only has one of these. These are written to using a subroutine (see "Address Modes" below).
- mem(opX) - The location in memory pointed to by opX (cannot be register or immediate address). Mem[mem(opX)] is equivalent to data(opX). These are stored in the effective address file.

Anywhere that "#decode X operands" is listed, jumps to a separate operate decoding routine, listed under "Addressing Modes."

Additionally, whenever "#write AUX to dest" is listed, another subroutine is called to do this. Also listed under "Addressing Modes."

Operation	RTL
	$PC \leftarrow Mem[MAR]$
fetch	$MAR \leftarrow PC$ $PC \leftarrow PC + 1$ $MDR \leftarrow Mem[MAR]$ $IR \leftarrow MDR$ #decode and goto instruction
halt	goto halt
alu2op (opc 10-1e)	# decode 3 operands $AUX \leftarrow data(op1)$ OP $data(op2)$ # write AUX to dest(dst) goto fetch
not	# decode 2 operands $AUX \leftarrow \overline{data(op1)}$ # write AUX to dest(dst) goto fetch
move	# decode 2 operands $AUX \leftarrow data(op1)$ # write AUX to dest(dst) goto fetch

pushs	$MT1 \leftarrow SP$
	$MAR \leftarrow MT1$
	$MT0 \leftarrow 0x08$
	while MT0 != 0x10 do:
	  $Mem[MAR] \leftarrow regfile[MT0]$
	  $MT1 \leftarrow MT1 + 4$
	  $MAR \leftarrow MT1$
	  $MT0 \leftarrow MT0 + 1$
	$AUX \leftarrow SP + 32$
	$SP \leftarrow AUX$
	goto fetch

pops	$MT1 \leftarrow SP$
	$MAR \leftarrow MT1$
	$MT0 \leftarrow 0x0f$
	while MT0 != 0x07 do:
	  $MDR \leftarrow Mem[MAR]$
	  $regfile[MT0] \leftarrow MDR$
	  $MT1 \leftarrow MT1 - 4$
	  $MAR \leftarrow MT1$
	  $MT0 \leftarrow MT0 - 1$
	$AUX \leftarrow SP - 32$
	$SP \leftarrow AUX$
	goto fetch

nop	goto fetch
fnop	goto nop
jump	# decode 1 operand $PC \leftarrow mem(op1)$ goto fetch
jal	# decode 1 operand $RA \leftarrow PC$ $PC \leftarrow mem(op1)$ goto fetch
beq	# decode 3 operands if data(op1) == data(op2): $PC \leftarrow mem(op3)$ goto fetch
bne	# decode 3 operands if data(op1) $\neq$ data(op2): $PC \leftarrow mem(op3)$ goto fetch
blt	# decode 3 operands if data(op1) < data(op2): $PC \leftarrow mem(op3)$ goto fetch
ble	# decode 3 operands if data(op1) $\leq$ data(op2): $PC \leftarrow mem(op3)$ goto fetch
Continued on the next page.	

Operation	RTL

addarr	# decode 4 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	MT2 ← mem(arr2)]
	AUX ← mem(arrdst) + data(len)
	MT3 ← AUX
	while MT0 < MT3 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  MT4 ← MDR
	  MAR ← MT2
	  MDR ← Mem[MAR]
	  AUX ← MT4 + MDR
	  MT4 ← AUX
	  MAR ← MT0
	  MDR ← MT4
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	  MT2 ← MT2 + 4
	goto fetch

saddarr	# decode 4 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	AUX ← mem(arrdst) + data(len)
	MT2 ← AUX
	MT3 ← data(op1)
	while MT0 < MT2 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  AUX ← MDR + MT3
	  MT4 ← AUX
	  MAR ← MT0
	  MDR ← MT4
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	goto fetch

subarr	# decode 4 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	MT2 ← mem(arr2)
	AUX ← mem(arrdst) + data(len)
	MT3 ← AUX
	while MT0 < MT3 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  MT4 ← MDR
	  MAR ← MT2
	  MDR ← Mem[MAR]
	  AUX ← MT4 − MDR
	  MT4 ← AUX
	  MAR ← MT0
	  MDR ← MT4
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	  MT2 ← MT2 + 4
	goto fetch

ssubarr	# decode 4 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	AUX ← mem(arrdst) + data(len)
	MT2 ← AUX
	MT3 ← data(op1)
	while MT0 < MT2 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  AUX ← MDR − MT3
	  MT4 ← AUX
	  MAR ← MT0
	  MDR ← MT4
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	goto fetch

Operation	RTL

dotvec	# decode 4 operands
	MT0 ← mem(arr1)
	MT1 ← mem(arr2)
	AUX ← mem(arr1) + data(len)
	MT2 ← AUX
	MT3 ← 0
	while MT0 < MT2 do:
	  MAR ← MT0
	  MDR ← Mem[MAR]
	  MT4 ← MDR
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  AUX ← MT4 × MDR
	  AUX ← MT3 + AUX
	  MT3 ← AUX
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	# write AUX to dest(dst)
	goto fetch

mularr	# decode 4 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	MT2 ← mem(arr2)
	AUX ← mem(arrdst) + data(len)
	MT3 ← AUX
	while MT0 < MT3 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  MT4 ← MDR
	  MAR ← MT2
	  MDR ← Mem[MAR]
	  AUX ← MT4 × MDR
	  MT4 ← AUX
	  MAR ← MT0
	  MDR ← MT4
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	  MT2 ← MT2 + 4
	goto fetch

smularr	# decode 4 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	AUX ← mem(arrdst) + data(len)
	MT2 ← AUX
	MT3 ← data(op1)
	while MT0 < MT2 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  AUX ← MDR × MT3
	  MT4 ← AUX
	  MAR ← MT0
	  MDR ← MT4
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	goto fetch

Operation	RTL

mulmat	# decode 6 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	MT2 ← mem(arr2)
	MT3 ← 0
	while MT3 < MT3 do:
	  MT4 ← 0
	  while MT4 < MT5 do:
	  MT5 ← 0
	  MT6 ← 0
	  while MT5 < MT4 do:
	  AUX ← MT3 × MT4
	  AUX ← AUX + MT5
	  AUX ← AUX × 4
	  AUX ← AUX + MT1
	  MT8 ← AUX
	  MAR ← MT8
	  MDR ← Mem[MAR]
	  MT7 ← MDR
	
	  AUX ← MT5 × MT5
	  AUX ← AUX + MT4
	  AUX ← AUX × 4
	  AUX ← AUX + MT2
	  MT11 ← AUX
	  MAR ← MT8
	  MDR ← Mem[MAR]
	  MT9 ← MDR
	  AUX ← MT10 × MT9
	  MT7 ← AUX
	
	  AUX ← MT6 + MT7
	  MT6 ← AUX
	  MT5 ← MT5 + 1
	  AUX ← MT3 × MT5
	  AUX ← AUX + MT4
	  AUX ← AUX × 4
	  AUX ← AUX + MT0
	  MT7 ← AUX
	  MAR ← MT7
	  MDR ← MT6
	  Mem[MAR] ← MDR
	  MT4 ← MT4 + 1
	  MT3 ← MT3 + 1
	goto fetch

copyarr	# decode 3 operands
	MT0 ← mem(arrdst)
	MT1 ← mem(arr1)
	AUX ← mem(arrdst) + data(len)
	MT2 ← AUX
	while MT0 < MT2 do:
	  MAR ← MT1
	  MDR ← Mem[MAR]
	  MAR ← MT0
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	  MT1 ← MT1 + 4
	goto fetch

setarr	# decode 3 operands
	MT0 ← mem(arrdst)
	AUX ← mem(arrdst) + data(len)
	MT1 ← AUX
	MT2 ← data(op1)
	while MT0 < MT1 do:
	  MAR ← MT0
	  MDR ← MT2
	  Mem[MAR] ← MDR
	  MT0 ← MT0 + 4
	goto fetch

Operation	RTL

plyeval	# decode 4 operands
	AUX ← mem(arr) + data(len)
	MT0 ← AUX
	MT1 ← mem(arr)
	MT2 ← data(x)
	MT3 ← 0
	while MT0 > MT1 do:
	  MT0 ← MT0 − 4
	  MAR ← MT0
	  MDR ← Mem[MAR]
	  AUX ← MT3 × MT2
	  AUX ← AUX + MDR
	  MT3 ← AUX
	# write AUX to dest(dst)
	dest(dst) ← MT3
	goto fetch

Addressing Modes

After the opcode byte is fetched and decoded, the instruction fetches and decodes each addressing mode.

To call the decode operand routine, for μA must be loaded with a 5 bit value featuring the following arguments:

1 bit	1 bit	3 bit
Must be mem?	Is dest?	Operand #

Then, the equivalent RTL to call decode is this

```
μA ← ARGUMENTS (see above)
μRA ← μPC
goto decode operand
```

Decode operand routine:

```
MAR ← PC
MDR ← Mem[MAR]
PC ← PC + 1
IR ← MDR
MT0 ← IR
AUX ← MT0 >> 5
if μA[3] == 1:
    DST ← 0
goto address mode's decode routine
```

Specific decode routines:

No.	Addressing mode	RTL

0	Register	`if μA[4] == 1:`
		`  halt (operand must be a memory location)`
		`if μA[3] == 1:`
		`  DST ← MT0[0:4]`
		`  if DST == 0:`
		`  halt (operand must be a destination)`
		`else:`
		`  orf[μA[0:2]] ← regfile[MT0[0:4]]`
		`μPC ← μRA`

1	Immediate	`if μA[3] == 1:`
		`  halt (operand must be a destination)`
		`if μA[4] == 1:`
		`  halt (operand must be a memory location)`
		`MAR ← PC`
		`MDR ← Mem[MAR]`
		`orf[μA[0:2]] ← MDR`
		`PC ← PC + 4`
		`μPC ← μRA`

2	Absolute	`MAR ← PC`
		`MDR ← Mem[MAR]`
		`PC ← PC + 4`
		`EA[μA[0:2]] ← MDR`
		`if μA[3] ≠ 1 and μA[4] ≠ 1:`
		`  MAR ← EA[μA[0:2]]`
		`  MDR ← Mem[MAR]`
		`  orf[μA[0:2]] ← MDR`
		`μPC ← μRA`

3	Displacement	`MAR ← PC`
		`MDR ← Mem[MAR]`
		`PC ← PC + 4`
		`AUX ← MDR + regfile[MT0[0:4]]`
		`EA[μA[0:2]] ← AUX`
		`if μA[3] ≠ 1 and μA[4] ≠ 1:`
		`  MAR ← EA[μA[0:2]]`
		`  MDR ← Mem[MAR]`
		`  orf[μA[0:2]] ← MDR`
		`μPC ← μRA`

Continues on Next Page		

No.	Addressing mode	RTL

4	Indexed	MT1 ← regfile[MT0[0:4]]
		MAR ← PC
		MDR ← Mem[MAR]
		MT0 ← MDR
		AUX ← regfile[MT0[0:4]] << 2
		AUX ← AUX + MT1
		EA[μA[0:2]] ← AUX
		if $\mu A[3] \neq 1$ and $\mu A[4] \neq 1$:
		  MAR ← EA[μA[0:2]]
		  MDR ← Mem[MAR]
		  orf[μA[0:2]] ← MDR
		$\mu PC \leftarrow \mu RA$

5	Memory Indirect	MAR ← PC
		MDR ← Mem[MAR]
		PC ← PC + 4
		MT1 ← MDR
		
		MAR ← regfile[MT0[0:4]]
		MDR ← Mem[MAR]
		AUX ← MT1 + MDR
		EA[μA[0:2]] ← AUX
		if $\mu A[3] \neq 1$ and $\mu A[4] \neq 1$:
		  MAR ← EA[μA[0:2]]
		  MDR ← Mem[MAR]
		  orf[μA[0:2]] ← MDR
		$\mu PC \leftarrow \mu RA$

6	Postincrement	EA[μA[0:2]] ← regfile[MT0[0:4]]
		regfile[MT0[0:4]] ← regfile[MT0[0:4]] + 4
		if $\mu A[3] \neq 1$ and $\mu A[4] \neq 1$:
		  MAR ← EA[μA[0:2]]
		  MDR ← Mem[MAR]
		  orf[μA[0:2]] ← MDR
		$\mu PC \leftarrow \mu RA$

7	Predecrement	regfile[MT0[0:4]] ← regfile[MT0[0:4]] − 4
		EA[μA[0:2]] ← regfile[MT0[0:4]]
		if $\mu A[3] \neq 1$ and $\mu A[4] \neq 1$:
		  MAR ← EA[μA[0:2]]
		  MDR ← Mem[MAR]
		  orf[μA[0:2]] ← MDR
		$\mu PC \leftarrow \mu RA$

To write to a dest(opX), first load the value to write into AUX. Then, load μA with a 3 bit (lower bits) number indicating which operand is the destination. Finally, run the following.

```
μA ← ARGUMENTS (see above)
μRA ← μPC
goto destwrite
```

destwrite routine:

```
if DST == 0:
    MAR ← EA[μA[0:2]]
    MDR ← AUX
    Mem[MAR] ← MDR
else:
    regfile[DST] ← AUX
μPC ← μRA
```

Micro Memory

Micro memory has 3 sections:

- 0x0000 - An opcode decoding table. This stores the locations of an opcode's routine in memory via a list of jump instructions. I.e. opc 0x8 would be the 9th item in this table.
- 0x0050 - An address mode decoding table. The same format as the opcode decoding table, just for address modes.
- 0x0060 - Microcode, which is filled with the format listed in the next section.

Microcode format

Microcode instructions use a hybrid encoding. Each instruction is 45 bits wide and is broken into three groups.

6 bits	20 bits	19 bits
Group 1	Group 2	Group 3

Group 1

Group 1 is used for incrementing and decrementing registers. It is 6 bits wide.

2 bits	4 bits
Amount	Register

Amount code	Increment by
0	+1
1	+4
2	-4
3	-1

Register code	Register
0	No operation
1-12	MT0 - MT11
13	PC
14	regfile[MT0[0:4]]
15	

Group 2

The layout of group 2 is tagged by the first two bits.

Tag	Operation
00	No operation
01	ALU operation
10	Data transfer
11	Memory access/special operations

Group 2: ALU operations

ALU operations are laid out as follows:

5 bits	5 bits	4 bits	4 bits
Operand 1	Operand 2	Operation	Padding

Operand code	Source
0-11	MT0-MT11
12-17	ORF[0-5]
18-23	EA[0-5]
24	PC
25	AUX
26	Constant 4
27	Constant 5

Code	Operation		Code	Operation
0	Add		8	AND
1	Subtract		9	OR
2	Multiply		10	XOR
3	Divide		11	NOT
4	Signed Divide		12	EQ
5	Shift left		13	< (Signed)
6	Shift right			
7	Signed shift right			

Group 2: Data transfers

Data transfer operations are laid out as follows. Empty spaces in the tables below indicate illegal transfers.

5 bits	5 bits	5 bits	3 bits
Data source	Data dest	Address source	Address dest

Code	Data source	Data dest
0-11	MT0-MT11	MT0-MT11
12-17	ORF[0-5]	ORF[0-5]
18-23	EA[0-5]	EA[0-5]
24	AUX	
25	MDR	MDR
26	regfile[MT0[0:4]]	regfile[MT0[0:4]]
27	IR	IR
28	Constant 0	DST
29	PC	orf[μA[0:2]]
30		EA[μA[0:2]]
31		regfile[DST]

Code	Addr source	Addr dest
0-5	EA[0-5]	EA[0-5]
6	PC	PC
7		MAR
8-19	MT0-MT11	
20	EA[μA[0:2]]	

Group 2: Memory access and special operations

Memory accesses and special operations are given by a single 4-bit field. Operations labeled debug are used to implement debug instructions.

4 bits	14 bits
Operation	Padding

Code	Operation
0	No operation
1	MDR $\leftarrow$ Mem[MAR]
2	PC $\leftarrow$ Mem[MAR]
3	Mem[MAR] $\leftarrow$ MDR
4	Dump value (debug)
5	Dump all registers (debug)
6	Dump memory range (debug)

Group 3

Group 3 is used for micro-control flow. The layout is as follows.

3 bits	16 bits
Operation	Address

Code	Operation
0	Special format
1	Unconditional jump
2	Branch if AUX == 0
2	Branch if AUX != 0
4	Branch if μA[3]
5	Branch if μA[4]
6	Jump and link
7	Branch if DST == 0

If the operation is 0 above, the address field follows the following format.

3 bits	8 bits	5 bits
Sub Operation	Padding	μA value

Address	Sub Operation
0	Halt normally
1	No operation (increment μPC)
2	Error: Operand is not a memory location
3	Error: Operand is not a destination
4	μPC $\leftarrow$ IR + opcode table offset
5	μPC $\leftarrow$ AUX + address mode table offset
6	μA $\leftarrow$ μA value, $\mu PC \leftarrow \mu PC + 1$
7	Return (μPC $\leftarrow$ μRA)